

ForeSight: Supplementary Material

Small Calibrated Decision Models as Opponent Forecasters for Lookahead Game Play

Saiakhil Chilaka, Cornell University

This supplement expands on the 2-page abstract. Section 1 maps every number in the paper to its source. All win rates carry 95% Wilson confidence intervals. The accompanying `code/` and `results/` folders contain the source code and the raw per-battle logs behind every table. Code: <https://github.com/akhilc08/aaai-27>.

1 Where each claim in the paper comes from

Paper claim	Details here
Forecasting accuracy, log-loss, calibration (Table 1)	Section 3, Table 1, Figures 1–2
56.5% vs Abyssal, $n=200$ (Table 2)	Section 4, Table 2
36–84 ms per forecast, 2.7–4.1 s per turn	Section 6, Table 4
Lookahead adds 13–20 points; depth 1 $\approx$ one-step 44%	Section 5, Table 3
Raw text hides a decisive lead (0.66 vs 0.87)	Section 2.1
ALFWorld 34.6% $\rightarrow$ 48.4%	Section 8, Table 5

2 Method details

2.1 State encoder and candidate actions

Each position is rendered from our side’s view as short precomputed facts, followed by the opponent’s recent history. Below is a real input from a logged turn against Abyssal (the model’s question asks which option the opponent will choose):

```
units alive: us 5, them 5; total HP: us 417%, them 378% (roughly even)
3 of their units not yet seen (counted at full HP)
our active 48%; their active 59% boosts def-1,spd-1; moves first: them
our best attack: 127% super-effective, hits to KO 1, KOs now;
  their best attack: 24% neutral, hits to KO 2
race: we knock out their active first; safe switches: us 1, them 0
our team: mudsdale fainted; heliolisk 70%; gothitelle 100%; heracross 100%;
  *tangrowth 48%; walrein 100%
their team: omastar 19%; skarmory fainted; *krookodile 59%; unseen; unseen; unseen
their previous actions (oldest first): switched to skarmory while threatened
  with a KO; used spikes (status move); used bravebird (their highest-damage
  option) while threatened with a KO; used earthquake (their highest-damage
  option); used closecombat (their highest-damage option)
our previous actions: attack, switch, attack

options:
```

```

o0: attack earthquake: ~13% of target HP, resisted, accuracy 100%
o1: attack closecombat: ~21% of target HP, neutral, accuracy 100%   <- actual
o2: attack unrevealed-ground-stab: ~12% of target HP, resisted, accuracy 100%
o3: attack unrevealed-dark-stab: ~24% of target HP, neutral, accuracy 100%
o4: switch in a unit at 19% HP that takes ~180% from the opposing
    active's best attack

```

The candidates are the opponent's revealed moves, one placeholder attack per type for unrevealed moves, and switches to Pokémon we have seen. About 5% of real turns (switches to never-seen Pokémon) have no matching candidate and are dropped from forecasting evaluation. There are about 3.3 candidates per turn on average, so uniform guessing scores about 33% top-1.

Why facts instead of raw text. In an earlier pilot with the zero-shot System One model Jev, a clearly winning position described in raw battle text moved its win-probability estimate only from 0.52 (the neutral starting position) to 0.66. The same position written as precomputed facts gave 0.87. The same model scored 16/20 on a type-matchup quiz, so the failure was in reading the state, not in lacking game knowledge.

2.2 Opponent forecaster

- **Base model:** Laya (convaiinnovations/laya), a 421M-parameter ModernBERT-large System One model that answers typed **choice** questions in one forward pass.
- **Data:** bot-vs-bot battles on a local Showdown server: 3,150 Gen 9 and 1,000 Gen 8 battles between SimpleHeuristics, MaxBasePower, Random and an HP-leaf search bot. Every decision of both sides is logged; the label is the opponent's actual next action mapped onto the candidate list. We sample 10k training decisions (from 142k logged). Splits are by battle.
- **Training:** top 4 of 28 encoder layers plus the decision head (75M trainable parameters, 18%), Laya's proper-scoring (log + spherical) objective, AdamW (encoder lr 2×10^{-5} , head lr 5×10^{-5} , weight decay 0.01), batch size 4, 100-step warmup then linear decay, gradient clipping 1.0, one epoch, bf16 autocast on Apple MPS, max input length 448 tokens. A temperature is then fitted on validation data. Training takes about 35 minutes on an Apple M5 Pro MacBook; peak memory about 9–10 GB.

2.3 Probability-weighted search

Depth-2 expectimax over an approximate simulator. At opponent nodes we keep replies until 90% of the forecast mass is covered (at most 3 at the root, 2 deeper). Chance nodes cover accuracy, damage rolls near a knock-out, secondary effects and speed ties, pruned to 90% mass (at most 4 outcomes) at the root and 75% (at most 2) deeper. At depth 1 our own actions are pruned to the 2 best by a free damage heuristic. Leaves are scored by the HP difference between the two sides. All forecast requests at one tree level are batched into a single model call.

Simulator approximations. Estimated random-battle stats; no abilities, items, weather, hazard damage, critical hits, Terastallization or volatile effects; unseen opponent Pokémon are generic placeholders and unrevealed moves are generic 90-power attacks of the user's type. These errors compound with depth (Section 5).

3 Forecasting results

Model	Evaluated on	n	Top-1	Log-loss	ECE
Uniform over candidates	Gen 9 bot test	4,000	.325	1.191	–
Always-strongest-attack rule	Gen 9 bot test	4,000	.486	1.159	–
Damage-softmax heuristic	Gen 9 bot test	4,000	.420	1.237	.097
Laya zero-shot	Gen 9 bot test	2,000	.434	1.148	.026
ForeSight (Gen 9)	Gen 9 bot test	4,000	.718	.702	.015
ForeSight (Gen 8)	Gen 8 bot test	3,754	.796	.542	.018
ForeSight (Gen 9)	Abyssal’s real moves	1,963	.730	.622	.041
ForeSight (Gen 8)	Abyssal’s real moves	1,963	.754	.582	.027
ForeSight (Gen 9)	Abyssal, 68 other battles	1,282	.753	.581	.024
ForeSight (Gen 8)	Abyssal, 68 other battles	1,282	.775	.540	.034

Table 1: Next-action forecasting on held-out turns. Abyssal never appears in training.

Breakdown (Gen 8 model). On held-out bot battles: top-1 .796, top-2 .941, top-3 .982. Accuracy falls with game length (turns 1–5: .89; 31+: .71) and with the number of options (2 options: .93 vs chance .50; 7+ options: .54 vs chance .14). On Abyssal’s real moves: top-1 .754, top-2 .935, top-3 .992. By opponent type (Gen 9 model): MaxBasePower .93, SimpleHeuristics .82, HP-leaf search bot .58, Random .41.

Known weakness: switches. Laya almost never ranks “switch” first, so top-1 accuracy on actual switches is 0 (they are rare among scorable turns: 120 of 3,754 bot turns and 10 of 1,963 Abyssal turns). The switch probabilities are still informative: P(switch) AUROC is 0.87 on bots and 0.86 on Abyssal.

Caveat. Revealed moves are listed in order of first use, so part of the accuracy on repetitive bots may come from that ordering.

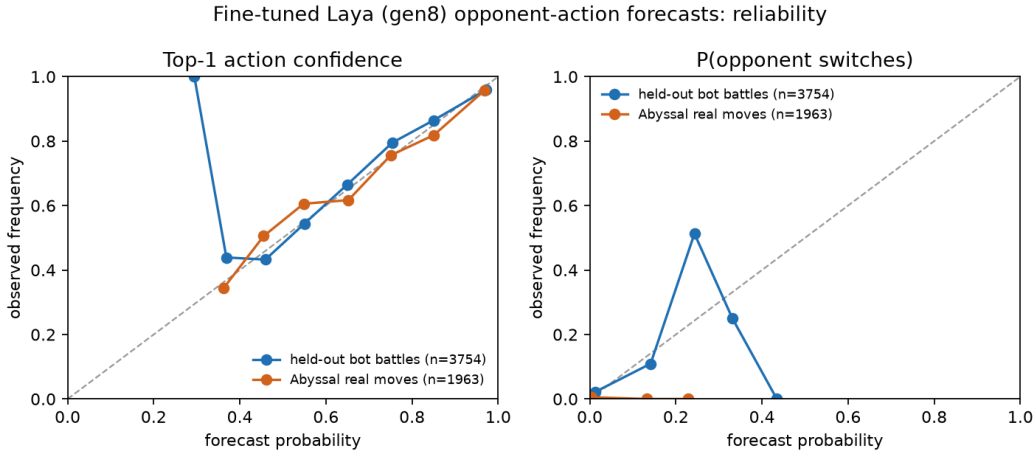


Figure 1: Reliability of the Gen 8 forecaster: top-1 confidence and P(switch).

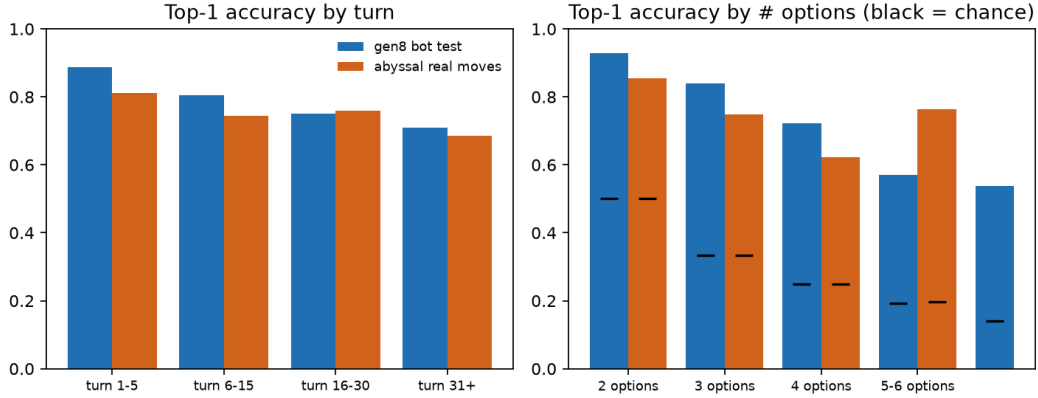


Figure 2: Gen 8 forecaster accuracy by turn number and by number of options.

4 Play results

All play uses PokéChamp’s protocol: Gen 8 random battles, Dynamax off, against PokéChamp’s real AbyssalPlayer, on a local Showdown server with a 15s turn limit. The Abyssal process is restarted every 20 battles because it leaks memory.

Agent	Opponent	Wins / n	Win rate [95% CI]
ForeSight, Gen 9 forecaster, depth 2 first 100 / second 100	Abyssal (Gen 8)	113 / 200 64 / 100, 49 / 100	56.5% [49.6–63.2]
ForeSight, Gen 8 forecaster, depth 2	Abyssal (Gen 8)	113 / 200	56.5% [49.6–63.2]
ForeSight, Gen 9 forecaster, depth 2	SimpleHeuristics (Gen 9)	121 / 200	60.5% [53.6–67.0]
<i>Published by PokéChamp (Karten et al. 2025), same protocol:</i>			
PokéChamp (GPT-4o)	Abyssal	–	70%
PokéChamp (Llama-3.1-8B)	Abyssal	–	64%
PokéLLMon (GPT-4o)	Abyssal	–	56%
One-step lookahead	Abyssal	–	44%

Table 2: Win rates. No ForeSight run used an LLM or any API.

5 Depth and opponent-model grid

To isolate the effect of lookahead, we fix the search settings and vary only the depth and the opponent model, against Abyssal. These runs use the search settings from an automated tuning loop (forecast temperature 0.3 and a small cost on our own switches; Section 7). The free opponent models make a turn take under 50 ms, so they allow 1,000 games per cell.

Opponent model	Depth 1	Depth 2	Depth 3
Laya (Gen 8)	40/119 = 33.6% [25.8–42.5]	105/200 = 52.5% [45.6–59.3]	too slow
Damage-softmax heuristic	424/1000 = 42.4% [39.4–45.5]	584/1000 = 58.4% [55.3–61.4]	518/1000 = 51.8% [48.7–54.9]
Uniform	422/1000 = 42.2% [39.2–45.3]	555/1000 = 55.5% [52.4–58.6]	–

Table 3: Win rate vs Abyssal by search depth and opponent model.

- **A second ply adds 13–20 points for every opponent model** (+18.9, +16.0 and +13.3). The heuristic and uniform differences are about 7 standard errors.

- **Depth 1 reproduces the published one-step baseline** (42% vs 44%), a check that our protocol matches PokéChamp’s.
- **Depth 3 is worse than depth 2** with the heuristic model. We attribute this to simulator errors compounding with depth; deeper search likely needs an exact engine.
- **The Laya depth-1 run** stopped at 119 games because an Abyssal process crashed inside a battle (a PokéChamp `AttributeError` in `_stat_estimation`).

6 Speed

Measurement (Apple M5 Pro, MPS)	Value
Laya forecast, single process, batch 1 / 4 / 16 / 64	84 / 45 / 39 / 36 ms per forecast
Forecasts that fit in a 15 s turn (batch 1 $\rightarrow$ 64)	$\sim 180 \rightarrow \sim 410$
Used by the depth-2 search per turn	~ 15 forecasts, ~ 73 nodes
Turn time, 4 concurrent battles sharing the GPU	mean 2.7–4.1 s; p95 4.5–7.1 s; max 11.0 s
Turns over the 15 s limit	0

Table 4: Latency. The search uses under a tenth of the turn budget.

7 Negative and null results

We report these so the main results can be read in context.

- **Better forecasts did not raise win rates.** The Gen 8 forecaster is 2.4 points more accurate on Abyssal than the Gen 9 one but wins at the same rate. On Abyssal’s real moves Laya is far more accurate than the damage-softmax heuristic (75.4% vs 48.9% top-1), yet at depth 2 the heuristic plays at least as well (Table 3), and even a uniform model reaches 55.5%. At this depth, the simulator and the HP leaf, not forecast accuracy, appear to be the bottleneck.
- **Automated tuning did not transfer.** An overnight loop screened 10 changes to the search and leaf against SimpleHeuristics (80–160 games each). Two were kept: forecast temperature 0.3 and a switch cost. On that practice opponent the tuned version reached 45.0% (131/291) vs 41.2% (66/160) for the baseline, which is not significant. On Abyssal, as a single held-out test, it scored 52.5% [45.6–59.3] vs 56.5% for the baseline: no gain. A richer leaf (Pokémon alive, status, matchup terms) changed win rates by at most 2.5 points in 1,500-game sweeps.
- **A hand-ported engine evaluator hurt.** A port of poke-engine’s position evaluator as the leaf scored 38.2% (26/68, stopped early), likely because it rewards boosts and status that our approximate simulator models poorly.
- **Zero-shot System One models forecast poorly.** Zero-shot Laya predicts 43.4% of bot actions (below the always-strongest-attack rule’s 48.6%), and as a win-probability forecaster its Brier score (.377) is worse than the base rate (.250).

8 ALFWorld pilot

An earlier pilot tested the same forecast-then-decide idea outside games. In ALFWorld (TextWorld, 134 unseen games), a cheap LLM (qwen3-30b-a3b) proposes its top 3 admissible actions and the System One model Jev picks the one it predicts most likely to make progress.

Comparison	Success	Units	Per-game difference [95% CI], p
LLM + Jev rerank (B) vs LLM top-1 (D), qwen3-30b	.484 vs .346	370 / 134 games	+.129 [+0.067, +.192], $p=.0001$
B vs LLM scoring its own options, qwen3-30b	.485 vs .412	369 / 134 games	+.068 [+0.014, +.123], $p=.019$
B vs D, larger qwen3-235b	.545 vs .493	402 / 134 games	+.052 [−.007, +.112], $p=.10$

Table 5: ALFWorld reranking. Units are game-seed runs; CIs from a cluster bootstrap over games; p from a per-game Wilcoxon test. Surprise-triggered replanning added nothing (C vs B: $-.006$).

The gain shrinks with a stronger LLM, which is expected: a better proposer leaves less to rerank.

9 Reproducibility

- **Hardware:** Apple M5 Pro MacBook (MPS). One job at a time; memory kept under 14 GB.
- **Software:** Python 3.11, torch 2.14, poke-env 0.16.1, layla 0.3.20, transformers 5.17, scikit-learn 1.9, Node 24. Pokémon Showdown at commit `a5df827`; PokéChamp at `0f84c46` (provides AbyssalPlayer).
- **Cost:** ForeSight makes no API calls. The Jev pilots (ALFWorld, encoder probe) used OpenRouter; every call’s cost is logged, about \$32 in total across all pilots in the log, including ones not reported here.
- **Included:** `code/` (ForeSight source, the tuning loop, the ALFWorld pilot), `results/` (per-battle logs for every run, aggregate results, forecasting scores). Not included: fine-tuned checkpoints (874 MB) and generated battle data, which the scripts regenerate. See `README.txt`.